

Forward and Backward Private Disjunctive Dynamic Searchable Symmetric Encryption With Leakage Suppression

Xiaojun Zhang¹, Lei Xu¹, Xingliang Yuan², *Senior Member, IEEE*, Yifeng Zheng³, Lin Mei¹,
and Chungen Xu¹

Abstract—Disjunctive dynamic searchable symmetric encryption (DDSSSE) is a promising technique that enables more expressive query capabilities over encrypted data while ensuring data security in dynamic update scenarios. While several schemes have been proposed to improve security and search efficiency, they still suffer from significant limitations, including high communication overhead and additional leakage during updates and queries. In this work, we initiate a formal study of disjunctive searchable symmetric encryption in the dynamic setting by refining its security model, including precise leakage functions and privacy notions. Based on this improved security notion, we propose a new DDSSSE scheme, called D²SSE, which achieves both forward and backward privacy while incurring low leakage. Specifically, starting from the static TWINSSE framework which reduces disjunctive keyword queries to conjunctive meta-keyword queries, we extend it to support dynamic updates while preserving correctness. At the same time, we simultaneously mitigate keyword-pair result pattern leakage and provide strong forward and backward privacy guarantees based on the combination of symmetric subset predicate encryption and puncturable encryption. Experimental results on the Wikimedia dataset show that D²SSE achieves a $0.25\times$ reduction in search latency compared to the state-of-the-art scheme DIEX. Moreover, D²SSE requires only about $0.06\times$ more client-side storage overhead of DIEX while maintaining comparable server-side storage cost.

Index Terms—Dynamic searchable symmetric encryption, disjunctive query, forward and backward privacy, keyword-pair result pattern.

I. INTRODUCTION

ENCRYPTED search has emerged as a critical primitive for secure cloud storage, enabling queries over encrypted data

without decryption and thereby preserving the confidentiality of both data and client queries [1]. Among a wide range of encrypted search algorithms, searchable symmetric encryption (SSE) [2], [3], [4] draws considerable attention due to its sublinear search complexity. Like plaintext search, SSE maintains an index on the server, but in encrypted form. A client holding the secret key generates a token for the desired keyword, which is then used to retrieve the corresponding encrypted data. An SSE scheme is called dynamic (DSSE) [5] if it supports additions and deletions.

Early DSSE schemes [6], [7], [8], [9] mainly handled single-keyword queries, yet real-world applications often demand richer expressiveness—for example, filtering emails by multiple sensitive terms, retrieving records within a score range, or finding documents related to any of several topics. A naive approach is to decompose complex queries into multiple single-keyword searches, encrypt and update each keyword independently, and then combine partial results via union, intersection, or filtering. While functional, this approach is inefficient and leaks the result set of each individual keyword query, exposing sensitive patterns to the server. Such leakage is not merely theoretical; recent leakage-abuse attacks [10], [11], [12] show that it can be exploited to infer query contents or even reconstruct datasets when combined with auxiliary knowledge.

To address these concerns, customized DSSE schemes have been proposed for more expressive queries while minimizing leakage. Among them, disjunctive queries, which retrieve documents matching any keyword from a given set, are especially useful in practical scenarios such as inclusive document filtering and alert-based monitoring. A pioneering solution, DIEX [13], leverages encrypted multi-maps to support disjunctive search with sublinear complexity, while achieving forward privacy, which prevents newly added documents from being linked to past queries. Subsequent work [14], [15], [16] has sought to improve the practicality of disjunctive SSE. However, most of these treat disjunctive queries merely as a special case of general Boolean queries, rather than as a core design goal.

One recent advance is TWINSSE by Bag et al. [15], which reduces storage overhead from quadratic to linear by introducing meta-keywords, disjunctions of carefully selected keywords, and transforming disjunctive queries into conjunctive meta-keyword queries that can be supported by existing conjunctive SSE schemes (e.g., OXT [17]). Despite this progress, TWINSSE

Received 4 October 2025; revised 21 May 2026; accepted 25 May 2026. Date of publication 1 June 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572243 and Grant 62202228 and in part by the Youth Science and Technology Talents Lifting Project of Jiangsu Association of Science and Technology under Grant JSTJ-2024-163. (Corresponding author: Lei Xu.)

Xiaojun Zhang, Lei Xu, Lin Mei, and Chungen Xu are with the School of Mathematics and Statistics, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: zhangxiaojun@njust.edu.cn; leixu@njust.edu.cn; meilin@njust.edu.cn; xuchung@njust.edu.cn).

Xingliang Yuan is with the School of Computing and Information Systems, The University of Melbourne, Parkville, VIC 3010, Australia (e-mail: xingliang.yuan@unimelb.edu.au).

Yifeng Zheng is with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong (e-mail: yifeng.zheng@polyu.edu.hk).

Digital Object Identifier 10.1109/TDSC.2026.3698827

remains inherently static, limiting its deployment in real-world scenarios. Moreover, recent studies [18], [19], [20] highlight that DIES suffers from keyword-pair result pattern (KPRP) leakage, which captures intersections between queried keywords. Likewise, TWINSSE reveals intersections between queried meta-keywords, as most existing conjunctive SSE schemes still leak KPRP.

Equally important, no existing disjunctive DSSE achieves backward privacy [9], which prevents the server from learning which documents were deleted after being added. Forward and backward privacy have become central security notions in DSSE, but current definitions for single-keyword settings fall short of capturing the unique challenges in disjunctive search. In particular, for disjunctive DSSE, backward privacy should ensure that the adversary learns only the update times of documents matching a disjunctive query, without linking them to specific keywords.

These limitations lead to a fundamental open question: *Can we design a disjunctive dynamic searchable symmetric encryption (DDSSE) scheme that supports efficient search, allows flexible updates, and simultaneously achieves both forward and backward privacy with reduced leakage?*

A. Overview of Contributions

In this work, we address the above challenges and propose the first forward and backward private DDSSE scheme that supports flexible dynamic updates and hides KPRP leakage, all while maintaining high search efficiency. Below, we provide a detailed description of our technical contributions.

Improving the security model of disjunctive DSSE: As mentioned, we first fill the privacy gap of update operations between a single query and a disjunctive query, specifically the Type III backward privacy. It is redefined to capture leakages unique to disjunctive queries: 1) within a single disjunctive query, the adversary learns only which documents currently match the query, when these documents were inserted, which deletions correspond to prior insertions, and the intersection pattern of any two queried keywords (i.e., KPRP); 2) across multiple disjunctive queries, the adversary learns only the query overlap pattern, which captures whether two queries share common keywords. As depicted above, unlike the Type III backward privacy of Bost et al. [9], our definition explicitly incorporates both KPRP leakage within a single query and query overlap pattern leakage across queries, two sources typically overlooked in previous backward privacy definitions. Moreover, regarding timing leakage, it restricts leakage to only the update times of documents matching the disjunctive query, without linking these updates to any specific keyword. The full formal definitions are given in Section III.

Devising a new forward and backward private DDSSE scheme with KPRP hiding: With the security notion above, now we show how to achieve them by taking TWINSSE[15] as our starting point. An intuitive idea is to substitute the conjunctive SSE scheme of TWINSSE with a forward and backward private conjunctive DSSE scheme (e.g. ODXT[21], SDSSE-CQ[22]). However, our analysis reveals that this naive solution (we refer

to it as TW_{ODXT}) suffers from the following problems. First, it introduces an erroneous deletion problem: when two keywords mapped to the same meta-keyword co-occur in a document, deleting the document with respect to one keyword incorrectly removes it from the set associated with the shared meta-keyword, causing subsequent queries on the other keyword to miss the document. Second, it leaks the number of meta-keywords associated with the updated keyword—a leakage we term the update volume pattern—which an adversary can use to further infer the updated keyword. Third, it still reveals intersections between queried meta-keywords due to residual KPRP leakage, referred to as Meta-KPRP, which enables known leakage-abuse attacks [11], [18], [23]. More details can be seen in Section IV.

To address the above problems, we design a new DDSSE construction, D^2SSE , that simultaneously achieves forward and backward privacy while concealing Meta-KPRP leakage. To remove erroneous deletions, we bind each keyword with its associated document identifiers and construct a mapping from meta-keyword to “keyword || identifier”, ensuring updates on different keywords mapped to the same meta-keyword do not interfere. To mitigate update volume leakage, we introduce dummy entries drawn from a predefined universe of meta-keywords to pad each update, thereby ensuring that all updates have the same volume. Importantly, using predefined meta-keywords for padding allows these dummies to potentially appear in future queries, making them indistinguishable from real entries. Finally, to simultaneously achieve backward privacy and conceal Meta-KPRP, we introduce three new data structures: EDB_T (storing encrypted documents with unique tags), EDB_D (recording punctured tags and its keys for deleted documents) and EDB_X (allowing private subset checks). With these structures, when a document is first inserted, it is stored in EDB_T along with its unique tag; if the document is later deleted, its tag are punctured and added to EDB_D , ensuring it will no longer appear in query results and thereby achieving backward privacy. Meanwhile, EDB_X allows the client to verify whether each surviving document also contains the other queried meta-keywords, without revealing which specific meta-keywords are matched, thus hiding Meta-KPRP. We also theoretically compare the complexity of D^2SSE with DIES[13], the state-of-the-art scheme in dynamic disjunctive search scenario, and analyze the security of D^2SSE . More details are provided in Sections V and VI.

Conducting a comprehensive performance evaluation: We conduct a detailed evaluation by comparing D^2SSE with DIES[13] and the naive solution TW_{ODXT} on two real-world datasets: the Enron Email and the Wikimedia Databases. Experimental results demonstrate that D^2SSE provides $0.25\times$ and $13\times$ faster search than DIES and TW_{ODXT} , respectively. It also reduces client-side storage to $0.06\times$ that of DIES while maintaining comparable server-side storage cost, and avoids the quadratic growth of communication overhead with query length. Further experimental results are given in Section VII.

B. Organization

The rest of the work is organized as follows. Section II reviews preliminaries and building blocks. Section III defines the syntax

and refined security model of DDSSE. Section IV presents a toy construction and its limitations. Section V details our refined scheme D²SSE and its theoretical analysis. Section VI provides the security proof, and Section VII reports performance evaluations. Finally, Section VIII discusses related work, and Section IX concludes the paper.

II. PRELIMINARIES

Notations: Let $\mathcal{W} = \{w_1, w_2, \dots, w_N\}$ be the keyword space. For any set S , we denote its cardinality simply by $|S|$. We model the database as a dictionary $\text{DB} = \{(\text{id}_i, W_i)\}_{i=1}^{|\text{DB}|}$, where each document $D_i \in \text{D}$ has a unique identifier id_i and is associated with a keyword set $W_i \subseteq \mathcal{W}$. For any keyword $w \in \mathcal{W}$, we define $\text{DB}(w) = \{\text{id}_i \mid w \in W_i\}$ as the set of all document identifiers containing w . For any disjunctive query $q = w_{\ell_1} \vee \dots \vee w_{\ell_q}$, we define $\text{DB}(q) = \bigcup_{t=1}^q \text{DB}(w_{\ell_t})$ as the set of all document identifiers containing at least one of the keywords in q . For integers a and b with $a \leq b$, we write $\llbracket a, b \rrbracket = \{a, a+1, \dots, b\}$, and abbreviate $\llbracket a \rrbracket = \{1, 2, \dots, a\}$. Finally, $\text{negl}(\lambda)$ denotes a function that is negligible in the security parameter λ .

A. Meta-Keyword Query Transformation Framework

A meta-keyword $mkw_{i,j} \in \{0,1\}^n$, introduced by Bag et al. [15], is a Boolean vector that represents excluding a contiguous block of keywords. Intuitively, it acts like a filter that “zeros out” keywords between positions i and j , while keeping all others active. Formally, for indices $i, j \in \llbracket n \rrbracket$ with $i \leq j$, we define $mkw_{i,j} = b_1 b_2 \dots b_n$, where $b_\ell = 0$ if $\ell \in \llbracket i, j \rrbracket$ and $b_\ell = 1$ otherwise. For example, if $n = 5$, then $mkw_{2,4} = 10001$. In particular, $mkw_{0,0} = 1^n$. The document set associated with a meta-keyword is given by $\text{DB}(mkw_{i,j}) = \bigcup_{\ell \in \llbracket n \rrbracket \setminus \llbracket i, j \rrbracket} \text{DB}(w_\ell)$.

Building on this notion, the TWINSSE framework [15] reduces a disjunctive keyword query into a collection of conjunctive meta-keyword queries. Specifically, let the keyword space be $\mathcal{W} = \{w_1, w_2, \dots, w_N\}$, partitioned into $n_B = \lceil N/n \rceil$ buckets of size n , where the k -th bucket contains $\mathcal{W}^{(k)} = \{w_{(k-1)n+1}, \dots, w_{kn}\}$. Then given a disjunctive query $q = w_{a_1} \vee w_{a_2} \vee \dots \vee w_{a_q}$, it is decomposed into n_B subqueries $q = \bigvee_{k=1}^{n_B} q^{(k)}$, where each $q^{(k)}$ only involves keywords from the k -th bucket and takes the form $q^{(k)} = w_{\ell_1}^{(k)} \vee w_{\ell_2}^{(k)} \vee \dots \vee w_{\ell_{q_k}}^{(k)}$ with $w_{\ell_j}^{(k)} \in \mathcal{W}^{(k)}$ for $j \in \llbracket q_k \rrbracket$.

After that, each subquery $q^{(k)}$ is then mapped into a conjunctive meta-keyword query, i.e.,

$$q_{mkw}^{(k)} = mkw_{i_0, j_0}^{(k)} \wedge mkw_{i_1, j_1}^{(k)} \wedge \dots \wedge mkw_{i_{q_k}, j_{q_k}}^{(k)},$$

where the indices (i_t, j_t) are chosen to exclude all keywords not appearing in $q^{(k)}$ according to the following rule:

$$mkw_{i_t, j_t}^{(k)} = \begin{cases} mkw_{\ell_t+1, \ell_{t+1}-1}^{(k)}, & \text{if } \ell_{t+1} > \ell_t + 1, \\ \emptyset, & \text{otherwise,} \end{cases}$$

with $\ell_0 \triangleq 0$. The last meta-keyword is set as

$$mkw_{i_{q_k}, j_{q_k}}^{(k)} = \begin{cases} \emptyset, & \text{if } q_k < n \text{ and } \ell_{q_k} = n, \\ mkw_{\ell_{q_k}+1, n}^{(k)}, & \text{if } q_k < n \text{ and } \ell_{q_k} < n, \\ mkw_{0,0}^{(k)}, & \text{if } q_k = n. \end{cases}$$

Finally, the meta-keyword query associated with q is obtained by combining all buckets: $q_{mkw} = \bigvee_{k=1}^{n_B} q_{mkw}^{(k)}$.

It should be noted that meta-keywords sharing the same index pair but belonging to different buckets are treated as distinct entities. To differentiate them, we prefix $n \cdot (t-1)$ zeros to the binary representation of meta-keywords in the t -th bucket. For example, under a bucket size of 8, the meta-keywords $mkw_{2,4}^{(1)}$ and $mkw_{2,4}^{(2)}$ are encoded as 10001111 and 0000000010001111, respectively. This bucket-aware encoding guarantees that meta-keywords with identical index pairs from different buckets do not collide during storage or query processing. Note that query processing under TWINSSE may return redundant results. In other words, besides all documents that correctly satisfy q , the output may also include a small number of irrelevant documents, denoted as,

$$\text{DB}(q) \subseteq \bigcup_{k \in \llbracket n_B \rrbracket} \text{DB}(q_{mkw}^{(k)}) = \bigcup_{k \in \llbracket n_B \rrbracket} \bigcap_{t \in \llbracket 0, q_k \rrbracket} \text{DB}(mkw_{i_t, j_t}^{(k)}).$$

B. Constrain Pseudorandom Function

Constrained Pseudorandom Function (CPRF) [24] is a variant of PRF that enables restricted evaluation under a specified constraint. Formally, given a security parameter $\lambda \in \mathbb{N}$, an input domain $\mathcal{X}$, an output domain $\mathcal{Y}$ and a constraint class $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$, where each $\mathcal{C}_\lambda : \mathcal{X} \rightarrow \{0,1\}$ is a Boolean circuit, a CPRF consists of the following algorithms:

- $msk \leftarrow \text{CPRF.KeyGen}(1^\lambda)$: It takes as input λ , and outputs a master secret key $msk \in \{0,1\}^\lambda$.
- $y \leftarrow \text{CPRF.Eval}(msk, x)$: It takes msk as input and a selected $x \in \mathcal{X}$, and outputs $y \in \mathcal{Y}$.
- $ck_C \leftarrow \text{CPRF.Con}(msk, C)$: It takes msk and a constraint circuit $C \in \mathcal{C}$ as input, and outputs a constraint key ck_C .
- $y/\perp \leftarrow \text{CPRF.CEval}(ck_C, x)$: It takes ck_C and an input $x \in \mathcal{X}$, and outputs $y \in \mathcal{Y}$ if $C(x) = 1$, otherwise $\perp$.

For any security parameter $\lambda \in \mathbb{N}$, constraint $C \in \mathcal{C}$, and input $x \in \mathcal{X}$, let $msk \leftarrow \text{CPRF.KeyGen}(1^\lambda)$ and $ck_C \leftarrow \text{CPRF.Con}(msk, C)$. Then, the correctness of the CPRF requires that $\text{CPRF.CEval}(ck_C, x) = \text{CPRF.Eval}(msk, x)$ if and only if $C(x) = 1$.

Security: A CPRF is secure if, even when an adversary is given multiple constrained keys and adaptive oracle access to function evaluations of its choice, the outputs of the PRF remain indistinguishable from those of a truly random function at all points the adversary cannot directly evaluate. Formally:

Definition 1: Let $\lambda \in \mathbb{N}$ be a security parameter. A function $F : \{0,1\}^\lambda \times \mathcal{X} \rightarrow \mathcal{Y}$ is a secure CPRF, if for any probabilistic polynomial time (PPT) adversary $\mathcal{A}$ with access to multiple constrained keys and adaptive access to function values at points of its choosing, it holds that

$$\text{Adv}_{F, \mathcal{A}}^{\text{CPRF}}(\lambda) = |\Pr[\mathcal{A}^{F(k, \cdot)}(1^\lambda) = 1] - \Pr[\mathcal{A}^{f(k, \cdot)}(1^\lambda) = 1]|$$

$$- \Pr[\mathcal{A}^{f(\cdot)}(1^\lambda) = 1] \leq \text{negl}(\lambda),$$

where $k \xleftarrow{\$} \{0, 1\}^\lambda$, $f : \mathcal{X} \rightarrow \mathcal{Y}$ is a truly random function.

C. Symmetric Encryption

A Symmetric Encryption (SE) scheme is defined as a tuple $\text{SE} = (\text{SE.Enc}, \text{SE.Dec})$, consisting of the following:

- $ct \leftarrow \text{SE.Enc}(k, m)$: It takes as input a secret key k and a plaintext m , and outputs a ciphertext ct .
- $m \leftarrow \text{SE.Dec}(k, ct)$: It takes as input the ciphertext ct and the secret key k , and outputs the plaintext m .

The correctness of an SE requires that for all plaintexts $m \in \mathcal{M}$ and all secret keys $k \in \mathcal{K}$, if $ct \leftarrow \text{SE.Enc}(k, m)$, then $\text{SE.Dec}(k, ct) = m$.

Definition 2: An SE scheme is Indistinguishability under Chosen-Plaintext Attack (IND-CPA) secure if for any PPT adversary $\mathcal{A}$, such that,

$$\text{Adv}_{\text{SE}, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \left| \Pr[\mathcal{A}(\text{SE.Enc}(k, m_0)) = 1] - \Pr[\mathcal{A}(\text{SE.Enc}(k, m_1)) = 1] \right| \leq \text{negl}(\lambda),$$

where the secret key $k \in \mathcal{K}$ remains hidden from $\mathcal{A}$, and $\mathcal{A}$ chooses two messages $m_0, m_1 \in \mathcal{M}$ of equal length and is allowed to make adaptive encryption queries, with the restriction that m_0 and m_1 are never included in these queries.

D. Symmetric Subset Predicate Encryption

Symmetric Subset Predicate Encryption (S^2PE) [20] allows a client holding a key associated with set X to decrypt ciphertexts under another set Y iff $X \subseteq Y$, enabling private subset membership testing. Specifically, an S^2PE scheme comprises the following algorithms:

- $(msk, \mathcal{M}, \mathcal{U}) \leftarrow \text{S}^2\text{PE.Setup}(1^\lambda)$: Taking as input the security parameter λ , it outputs a master secret key msk , a plaintext space $\mathcal{M}$, and an attribute universe $\mathcal{U}$.
- $sk_X \leftarrow \text{S}^2\text{PE.KeyGen}(msk, X)$: Taking as input msk and predicate set $X \subseteq \mathcal{U}$, it outputs decryption key sk_X .
- $ct \leftarrow \text{S}^2\text{PE.Enc}(msk, m, Y)$: Taking as input msk , $m \in \mathcal{M}$, and an attribute set $Y \subseteq \mathcal{U}$, it outputs ciphertext ct .
- $m / \perp \leftarrow \text{S}^2\text{PE.Dec}(sk_X, ct)$: Taking as input sk_X and ct , it outputs the plaintext m if $X \subseteq Y$, and $\perp$ otherwise.

The correctness of an S^2PE scheme requires that decryption succeeds if and only if the predicate condition is satisfied.

Security: Informally, the security of S^2PE requires that a ciphertext leaks nothing about the underlying plaintext or its associated attribute set beyond what is implied by the subset predicate. Following the work of Wang et al. [20], the formal notion is given below.

Definition 3: Let S^2PE be an S^2PE scheme and let $\mathcal{L}$ be a leakage function that specifies the information revealed to the adversary during the scheme execution. We define the real-world and ideal-world games $\text{S}^2\text{PE}_{\text{Real}}^{\mathcal{A}}$, $\text{S}^2\text{PE}_{\text{Ideal}}^{\mathcal{A}, \mathcal{S}}$ as follows:

- $\text{S}^2\text{PE}_{\text{Real}}^{\mathcal{A}}(\lambda)$: A PPT adversary $\mathcal{A}$ interacts with the real S^2PE system and its view consists of the ciphertexts and transcripts generated by honest parties.

- $\text{S}^2\text{PE}_{\text{Ideal}}^{\mathcal{A}, \mathcal{S}}(\lambda)$: A PPT adversary $\mathcal{A}$ interacts with a simulator $\mathcal{S}$ that has access only to $\mathcal{L}$, and its view consists of simulated ciphertexts and transcripts without revealing plaintexts.

We say that S^2PE is $\mathcal{L}$ -selectively-secure against adaptive attacks, if for any PPT adversary $\mathcal{A}$, there exists an efficient simulator $\mathcal{S}$ such that,

$$\text{Adv}_{\mathcal{L}, \mathcal{A}}^{\text{S}^2\text{PE}}(\lambda) = \left| \Pr[\text{S}^2\text{PE}_{\text{Real}}^{\mathcal{A}}(\lambda) = 1] - \Pr[\text{S}^2\text{PE}_{\text{Ideal}}^{\mathcal{A}, \mathcal{S}}(\lambda) = 1] \right| \leq \text{negl}(\lambda).$$

E. Incremental Puncturable Encryption

Let $\mathcal{M}, \mathcal{T}$ denote the plaintext space and tag space, respectively. An Incremental Puncturable Encryption (IPE) [9] scheme is composed of the following algorithms:

- $(pk, SK_0) \leftarrow \text{IPE.KeyGen}(1^\lambda)$: Taking as input a security parameter λ and outputs a public key pk and an initial secret key $SK_0 = \{sk_0\}$.
- $ct \leftarrow \text{IPE.Enc}(pk, m, t)$: Taking as input $pk, m \in \mathcal{M}$, and a tag $t \in \mathcal{T}$, it outputs a ciphertext ct .
- $SK_{i+1} \leftarrow \text{IPE.Punc}(SK_i, t)$: Taking as input $SK_i = (sk_0, sk_1, \dots, sk_i)$, $t \in \mathcal{T}$, it computes $(sk'_0, sk_{i+1}) \leftarrow \text{IPE.IncP}(sk_0, t)$ and outputs a new secret key $SK_{i+1} = (sk'_0, sk_1, \dots, sk_i, sk_{i+1})$. The key SK_{i+1} retains the decryption capability of SK_i for all ciphertexts except those encrypted under the puncturable tag t .
- $m / \perp \leftarrow \text{IPE.Dec}(SK_i, ct, t)$: Taking as input SK_i , $t \in \mathcal{T}$, it outputs a plaintext $m \in \mathcal{M}$ or $\perp$ if decryption fails.

An IPE scheme is considered correct if, for any tag t , a ciphertext c generated by encrypting a plaintext m under t can always be correctly decrypted using a secret key punctured on any set of tags excluding t .

Security: The security of IPE requires that even if an adversary obtains some punctured secret keys and can adaptively query the tags corresponding to these keys, it still cannot distinguish the ciphertexts of any two messages encrypted under an unpunctured tag. Formally,

Definition 4: An IPE scheme is IND-PUN-CPA secure if for any PPT adversary $\mathcal{A}$, such that,

$$\text{Adv}_{\text{IPE}, \mathcal{A}}^{\text{PUN-CPA}}(\lambda) = \left| \Pr[\mathcal{A}(\text{IPE.Enc}(pk, m_0, t)) = 1] - \Pr[\mathcal{A}(\text{IPE.Enc}(pk, m_1, t)) = 1] \right| \leq \text{negl}(\lambda),$$

where the tag t of the challenge ciphertext $\text{IPE.Enc}(pk, m_b, t)$ does not belong to any previously punctured set.

III. SYNTAX AND REFINED SECURITY MODEL

In this section, we will present the formal definition of disjunctive dynamic searchable symmetric encryption (DDSSE) along with refined security models for backward privacy.

A. The Syntax of DDSSE

Definition 5: A DDSSE scheme D^2SSE consists of one algorithm and two protocols between the client and the server, which operate as follows:

- $(K, \sigma, EDB) \leftarrow D^2SSE.Setup(1^\lambda)$: A probabilistic algorithm that takes as input the security parameter λ , and outputs a secret key K and a client state σ , together with an encrypted database EDB for the server.
- $(\sigma', EDB') \leftarrow D^2SSE.Update(K, \sigma, (op, (w, id)), EDB)$: An interactive protocol between the client and the server. The client inputs the secret key K , its local state σ , and an update request $(op, (w, id))$, where $op \in \{add, del\}$ specifies whether to add or delete the association of keyword w with document identifier id . The server inputs the encrypted database EDB and outputs an updated client state σ' and an updated encrypted database EDB' .
- $R \leftarrow D^2SSE.Search(K, \sigma, q, EDB)$: An interactive protocol where the client inputs (K, σ) and a disjunctive query $q = w_1 \vee w_2 \vee \dots \vee w_q$, and the server inputs EDB . The protocol returns to the client the set of document identifiers R that contain at least one of the keywords in the disjunctive query.

A DDSSE scheme is considered correct if, for any database DB and any disjunctive query q , $D^2SSE.Search$ protocol returns the complete set of documents matching q .

B. The Security of DDSSE

A DDSSE is secure if the adversary cannot learn anything encrypted dataset and queries beyond the explicit leakage parameterized by a stateful function. Let $\mathcal{L} = (\mathcal{L}_{Setup}, \mathcal{L}_{Update}, \mathcal{L}_{Search})$ denote the information leaked during the setup, update, and search stages. Following [3], we give the security notion of DDSSE via the real and ideal experiments:

- $D^2SSE_{Real}^A(\lambda)$: $\mathcal{A}$ first selects a database DB . The game then executes $D^2SSE.Setup$ to initialize the encrypted database EDB and sends EDB to $\mathcal{A}$. Then $\mathcal{A}$ can adaptively issue a sequence of disjunctive queries qs (resp. update for (w, id)). For each q (resp. $(op, (w, id))$), the game runs the $D^2SSE.Search$ (resp. $D^2SSE.Update$) protocol with client input (K, σ) and server input EDB , returning both the search (resp. update) transcript and the client output to $\mathcal{A}$. Finally, $\mathcal{A}$ outputs a bit, which is returned as the output of the real-world game.
- $D^2SSE_{Ideal}^{A,S}(\lambda)$: $\mathcal{A}$ also selects a database DB . The game then runs $EDB \leftarrow \mathcal{S}(\mathcal{L}_{Setup})$ and sends EDB to $\mathcal{A}$. $\mathcal{A}$ can adaptively issue a sequence of disjunctive queries (resp. update operations). For each query (resp. update), the game responds with a simulated transcript generated by $\mathcal{S}$ based on the leakage $\mathcal{L}_{Search}$ (resp. $\mathcal{L}_{Update}$). Finally, $\mathcal{A}$ outputs a bit, which is returned as the output of the ideal-world game.

Definition 6: We say that D^2SSE is $\mathcal{L}$ -adaptively-secure against adaptive attacks, if for any PPT adversary $\mathcal{A}$, there exists an efficient simulator $\mathcal{S}$ such that,

$$\left| \Pr [D^2SSE_{Real}^A(\lambda)=1] - \Pr [D^2SSE_{Ideal}^{A,S}(\lambda)=1] \right| \leq \text{negl}(\lambda).$$

Forward and Backward Privacy: Beyond the above, DSSE are also required to meet the forward and backward privacy, two fundamental security properties that limit the information leaked during database updates. Briefly, forward privacy ensures that newly added documents cannot be linked to past search queries, while backward privacy requires that deleted documents cannot be retrieved by future queries.

Before presenting the formal notion, we first introduce several functions related to the leakage profiles. Consider a database DB and a query list $\mathcal{Q} = \{(\mu, op, (w, id))\}$ or $\{(\mu, q = w_1 \vee \dots \vee w_q)\}$. Here, $(\mu, op, (w, id))$ denotes an update query with timestamp μ and (μ, q) denotes a disjunctive search query with timestamp μ . The above mentioned leakage functions are defined as follows:

- $qrp(q)$ is a tuple $(TimeDB(q), krp(q))$, which is called the query result pattern. Here, $TimeDB(q)$ denotes the set of all documents matching the disjunctive query q , excluding those deleted prior to the query, together with the timestamps of their insertions into the database. $krp(q)$ denotes the set of pairwise intersections of keyword document sets $\{DB(w_i) \cap DB(w_j)\}$. Formally,

$$TimeDB(q) = \bigcup_{i=1}^q \{(\mu, id) \mid (\mu, add, (w_i, id)) \in \mathcal{Q} \text{ and } \forall \mu', (\mu', del, (w_i, id)) \notin \mathcal{Q}\},$$

$$krp(q) = \{DB(w_i) \cap DB(w_j)\}_{1 \leq i < j \leq q}.$$

- $DelHist(q)$ denotes the deletion history, defined as the set of all deletion timestamps for the keywords in query q together with the timestamps of the corresponding insertions they remove. Formally,

$$DelHist(q) = \bigcup_{i=1}^q \{(\mu^{add}, \mu^{del}) \mid \exists id, \text{ s.t. } (\mu^{del}, del, (w, id)) \in \mathcal{Q} \text{ and } (\mu^{add}, add, (w, id)) \in \mathcal{Q}\}.$$

- qop is the query overlap pattern which indicates whether two disjunctive queries $q^{(i)}$ and $q^{(j)}$ share any common keyword. Formally,

$$qop[i, j] = \begin{cases} 1 & \text{if } q^{(i)} \cap q^{(j)} \neq \emptyset, \\ 0 & \text{otherwise.} \end{cases}$$

Building on the above leakage functions, we formally define forward and backward privacy as follows.

Definition 7 (Forward Privacy): An $\mathcal{L}$ -adaptively-secure DDSSE scheme is forward privacy, if the update leakage function $\mathcal{L}_{Update}$ satisfy:

$$\mathcal{L}_{Update}(op, (w, id)) = \mathcal{L}'(op),$$

where $\mathcal{L}'$ is a stateless function.

Definition 8 (Backward Privacy): An $\mathcal{L}$ -adaptively-secure DDSSE scheme is Type III backward privacy, if update and search leakage functions $\mathcal{L}_{Update}, \mathcal{L}_{Search}$ can be written as:

$$\mathcal{L}_{Update}(op, (w, id)) = \mathcal{L}'(op),$$

$$\mathcal{L}_{Search}(q) = \mathcal{L}''(qrp(q), DelHist(q), qop),$$

where $\mathcal{L}'$ and $\mathcal{L}''$ are two stateless functions.

Remark: Note that, our backward privacy introduces additional leakage profiles—KPRP leakage within a single query and query overlap pattern leakage across queries—which are usually overlooked in prior works. Usually, existing work [9] classifies backward privacy into three levels: Type III, Type II, and Type I. Each provides progressively stronger guarantees against leakage from deleted documents in future queries. We focus on the first one in this work.

IV. OUR TOY CONSTRUCTION

Now we give a toy construction by extending TWINSSE [15] to support dynamic updates, serving as a naive DDSSE scheme for illustration. We will show that this baseline suffers from several limitations, including erroneous deletions, update volume pattern leakage, and Meta-KPRP leakage, thereby motivating the need for a more advanced construction.

A. A Toy Construction

The Construction: To support dynamic updates in disjunctive SSE, a natural idea is to replace the static conjunctive SSE scheme in TWINSSE[15] with a forward and backward private conjunctive DSSE scheme (e.g., ODXT[21], SDSSE-CQ[22]). In essence, for each keyword-identifier pair to be updated, the client locally computes all associated meta-keyword and identifier pairs, and then independently performs updates on each of them by invoking these conjunctive DSSE schemes. For clarity, we refer to the toy solution that uses ODXT as $\text{TWIN}_{\text{ODXT}}$. Specifically, given a pair (w_ℓ, id) , $\text{TWIN}_{\text{ODXT}}$ performs the following steps to obtain data update.

- *Step 1:* The client computes the set of all meta-keywords, $\{mkw_{i,j}\}_{\ell \notin [i,j]}$, associated with the keyword w_ℓ , and then constructs the corresponding set of all meta-keyword and identifier pairs, $\{(mkw_{i,j}, \text{id})\}_{\ell \notin [i,j]}$, to be updated.
- *Step 2:* For each meta-keyword and identifier pair, the client and the server collaboratively execute the protocol $\text{ODXT.Update}(K, \sigma, \text{op}, (mkw_{i,j}, \text{id}), \text{EDB})$ to complete update, where $K, \sigma, \text{op}, \text{EDB}$ are secret key, secret state, update operation and encrypted database, respectively.

For search, a disjunctive keyword query is first reduced to a conjunctive meta-keyword query, after which the client and server jointly execute ODXT.Search to retrieve the results. Notably, this procedure directly follows the TWINSSE framework and is therefore not our focus here.

B. Limitations of the Toy Construction

While the toy construction is simple and illustrative, it suffers from three inherent problems: erroneous deletions, update volume pattern leakage, and Meta-KPRP leakage. We elaborate on each of these issues below.

Erroneous Deletion Problem: The first issue is $\text{TWIN}_{\text{ODXT}}$ may incur erroneous deletion. Specifically, because different keywords may map to the same meta-keyword, deleting one keyword may unintentionally impact another keyword that shares

the same meta-keyword but has not been updated, thereby causing erroneous deletions. A detailed illustration is provided in Example 1.

Example 1: Suppose $N = 4$, and the set of document identifiers associated with each keyword are:

$$\begin{aligned} \text{DB}(w_1) &= \{\text{id}_1, \text{id}_3\}, & \text{DB}(w_2) &= \{\text{id}_1, \text{id}_2, \text{id}_5\}, \\ \text{DB}(w_3) &= \{\text{id}_1, \text{id}_3, \text{id}_4\}, & \text{DB}(w_4) &= \{\text{id}_1, \text{id}_2, \text{id}_4, \text{id}_6\}. \end{aligned}$$

Then, the document sets corresponding to the relevant meta-keywords are computed as (due to space limitations, only the necessary parts are shown):

$$\begin{aligned} \text{DB}(mkw_{1,1}) &= \bigcup_{i=2}^4 \text{DB}(w_i) = \{\text{id}_1, \text{id}_2, \text{id}_3, \text{id}_4, \text{id}_5, \text{id}_6\}, \\ \text{DB}(mkw_{4,4}) &= \bigcup_{i=1}^3 \text{DB}(w_i) = \{\text{id}_1, \text{id}_2, \text{id}_3, \text{id}_4, \text{id}_5\}. \end{aligned}$$

Now, if the client performs two sequential deletion operations, $(\text{del}, (w_1, \text{id}_1))$ and $(\text{del}, (w_4, \text{id}_1))$, id_1 will be removed from both $\text{DB}(mkw_{4,4})$ and $\text{DB}(mkw_{1,1})$. Consequently, when a subsequent query $q = w_2 \vee w_3$ is issued, the server evaluates the corresponding conjunctive query $q_{mkw} = mkw_{1,1} \wedge mkw_{4,4}$. However, id_1 is no longer included in the result set, which leads to an erroneous deletion.

Update Volume Pattern Leakage: $\text{TWIN}_{\text{ODXT}}$ also suffers from update volume pattern leakage during the update procedure. Specifically, when processing an update for (w, id) , $\text{TWIN}_{\text{ODXT}}$ enumerates all meta-keyword/identifier pairs associated with w and updates them individually. This allows the server to learn the number of meta-keywords involved in each update, leading to what we refer to as update volume pattern leakage. Importantly, this leakage is keyword-dependent: different keywords are associated with different numbers of meta-keywords. We denote by $\|w_i\|$ the number of meta-keywords associated with keyword w_i , which is given by

$$\|w_i\| = 1 - i(N+1-i) + \sum_{j=0}^N j = \frac{N(N+1)}{2} - i(N+1-i) + 1.$$

Therefore, the server can infer the original keyword being updated by counting the number of meta-keywords involved in the update stage. A detailed example is presented below.

Example 2: Supposing $N = 5$, the meta-keywords corresponding to the keyword w_3 are $mkw_{1,1}, mkw_{1,2}, mkw_{2,2}, mkw_{4,4}, mkw_{4,5}, mkw_{5,5}, mkw_{0,0}$ —a total of 7 entries, i.e., $\|w_3\| = 7$. Thus, updating w_3 requires 7 meta-keyword updates. Similarly, $\|w_1\| = \|w_5\| = 11$ and $\|w_2\| = \|w_4\| = 8$. Therefore, the server can observe the number of meta-keyword updates in a single keyword update to infer the keyword value.

Meta-KPRP Leakage: The last issue is Meta-KPRP leakage. To the best of our knowledge, nearly all existing forward and backward private conjunctive DSSE schemes (e.g., ODXT[21], SDSSE-CQ[22]) leak the KPRP. When such schemes are directly integrated into the TWINSSE framework, this leakage propagates to the meta-keyword level, revealing intersections between queried meta-keywords—referred to as Meta-KPRP.

Algorithm 1: D²SSE.Setup(1^λ).

Input: A security parameter λ
Output: A secret key K and a secret state σ to client, an encrypted database EDB to server

- 1: $K'_E, K''_E, K_T, K_X, K_M \xleftarrow{\$} \{0, 1\}^\lambda$ for PRF F
- 2: $msk \leftarrow S^2PE.Setup(1^\lambda)$
- 3: $K \leftarrow (K'_E, K''_E, K_T, K_X, K_M, msk)$
- 4: **ACT**, **DCT**, **SK** $\leftarrow$ empty map
- 5: $\sigma \leftarrow (\mathbf{ACT}, \mathbf{DCT}, \mathbf{SK})$
- 6: **EDB_T**, **EDB_D**, **EDB_X**, **EDB_{cache}** $\leftarrow$ empty map
- 7: **EDB** $\leftarrow (\mathbf{EDB}_T, \mathbf{EDB}_D, \mathbf{EDB}_X, \mathbf{EDB}_{cache})$
- 8: **return** $K, \sigma, \mathbf{EDB}$

As a result, known leakage-abuse attacks exploiting KPRP [11], [18], [23] remain applicable and can compromise the privacy of disjunctive queries.

V. NON-LOSSY SOLUTION WITH LEAKAGE SUPPRESSION

Due to the limitations identified above, we thus present a refined scheme, D²SSE, which resolves erroneous deletions, conceals both update volume and Meta-KPRP leakage, and preserves forward and backward privacy. The detailed construction is given below.

A. Our Construction

System Setup: As shown in Algorithm 1, the setup phase starts by randomly selecting some secret keys $K'_E, K''_E, K_X, K_S, K_T$ for PRF F and generating a master secret key msk for S²PE (lines 1-3). The client then initializes the system state, including maps **ACT** and **DCT** to track the addition and deletion counters of meta-keywords, and **SK** to store IPE keys (lines 4-5). In addition, four empty maps, **EDB_T**, **EDB_X**, **EDB_D** and **EDB_{cache}**, are initialized. Specifically, **EDB_T** is designed to store encrypted documents along with unique tags; **EDB_D** will be used to maintaining keys and punctured tags for deleted documents; **EDB_X** is intended to record indicators of meta-keyword co-occurrence within the same document; and **EDB_{cache}** will act as a cache for search results. (lines 6-7) Finally, the client securely retains all secret keys and the update states, while the server stores the encrypted database EDB for use in subsequent queries and updates (line 8).

Update: When a new tuple $(op, (w_\ell, id))$ arrives, the client and the server jointly run Algorithm 2 to perform the update. The client first computes the bucket index t and identifies the meta-keyword set M associated with w_ℓ in that bucket (lines 1-2). To conceal update volume leakage, the client constructs a padded set $M' = \text{Pad}(M, M^*)$, where $\text{Pad}(M, M^*)$ randomly samples elements from M^* and inserts them into M until the target size is reached (lines 3-4). The padding size is determined by the maximum meta-keyword cardinality within the bucket following strategy in previous single keyword search schemes [25]. Specifically, the number of meta-keywords for the i -th keyword is $\|w_i\| = \frac{n(n+1)}{2} - i(n+1-i) + 1$. Since $i(n+1-i)$ is maximized at the center and minimized at the boundaries, the largest set size occurs at $i = 1$ or $i = n$,

Algorithm 2: D²SSE.Update($K, \sigma, op, (w_\ell, id), \mathbf{EDB}$).

1: **Input:** The secret key K , the secret state σ , an update tuple $(op, (w_\ell, id))$ and the encrypted database EDB
2: **Output:** An updated state σ' to client and a modified encrypted database EDB' to the server
3: *Client:*
1: $t \leftarrow \lceil \frac{\ell}{n} \rceil, b \leftarrow \ell - n(t-1)$
2: $M \leftarrow \{mkw_{i,j}^{(t)}\}_{b \in \llbracket n \rrbracket \setminus \llbracket i, j \rrbracket} \cup \{mkw_{0,0}^{(t)}\}$
3: $M^* \leftarrow \{mkw_{i,j}^{(t)} \mid i \leq j, b \in \llbracket i, j \rrbracket\}$
4: $M' \leftarrow \text{Pad}(M, M^*)$, s.t. $|M'| = \frac{n(n-1)}{2} + 1$
5: **for** $mkw \in M'$ **do**
6: $tag \leftarrow F(K_T, mkw \| w_\ell \| id)$
7: $sk \leftarrow \mathbf{SK}[mkw]$
8: **if** $sk = \perp$ **then**
9: $sk \leftarrow \text{IPE.KeyGen}(1^\lambda), \mathbf{SK}[mkw] \leftarrow sk$
10: **end if**
11: **if** $op = \text{add}$ **then**
12: $c \leftarrow \mathbf{ACT}[mkw]$
13: **if** $c = \perp$ **then**
14: $c \leftarrow 0, \mathbf{ACT}[mkw] \leftarrow c$
15: **end if**
16: $loc \leftarrow F(K_M, mkw \| c)$
17: $[UT_{mkw}^c, e] \leftarrow \text{TUPd}(K'_E, (w, id), mkw, c, M^*)$
18: $cip \leftarrow \text{IPE.Enc}(sk, e \| loc, tag), e_T \leftarrow [cip, tag]$
19: **for** $mkw' \in M' \setminus \{mkw\}$ **do**
20: $xtag \leftarrow F(K_X, mkw \| mkw' \| c)$
21: $\mathbf{XSet} \leftarrow \mathbf{XSet} \cup \{xtag\}$
22: **end for**
23: $\mathbf{ct}_X \leftarrow S^2PE.\text{Enc}(msk, \text{"True"}, \mathbf{XSet})$
24: Send $[UT_{mkw}^c, e_T], [loc, \mathbf{ct}_X]$ to Server
25: **else** $\triangleright op = \text{del}$
26: $c \leftarrow \mathbf{DCT}[mkw]$
27: **if** $c = \perp$ **then**
28: $c \leftarrow 0, \mathbf{DCT}[mkw] \leftarrow c$
29: **end if**
30: $UT_{mkw}^c \leftarrow \text{TUPd}(K''_E, (w, id), mkw, c, M^*)$
31: $(sk', sk_p) \leftarrow \text{IPE.IncPunc}(sk, tag)$
32: $\mathbf{SK}[mkw] \leftarrow sk'$
33: Send $[UT_{mkw}^c, e_D = (sk_p, tag)]$ to Server
34: **end if**
35: **end for**
Server:
36: **for** $mkw \in M'$ **do**
37: $\mathbf{EDB}_T[UT_{mkw}^c] \leftarrow e_T, \mathbf{EDB}_D[UT_{mkw}^c] \leftarrow e_D$
38: $\mathbf{EDB}_X[loc] \leftarrow \mathbf{ct}_X$
39: **end for**
40: **return** $\sigma', \mathbf{EDB}'$
41: **function** $\text{TUPd}(K_E, (w, id), mkw, c, M^*)$
42: **if** $mkw \in M^*$ **then**
43: $sig \leftarrow 0 \quad \triangleright$ dummy meta-keyword
44: **else**
45: $sig \leftarrow 1 \quad \triangleright$ real meta-keyword
46: **end if**
47: $K_{mkw} \| \tilde{K}_{mkw} \leftarrow F(K_E, mkw)$
48: $T_{mkw}^c \leftarrow \text{CPRF.Eval}(K_{mkw}, c)$
49: $UT_{mkw}^c \leftarrow H_1(\tilde{K}_{mkw}, T_{mkw}^c)$
50: $xind \leftarrow F(K_{mkw}, id)$
51: $y \leftarrow \text{SE.Enc}(K_{mkw}, w \| id \| sig)$
52: $e \leftarrow (y \| xind) \oplus H_2(\tilde{K}_{mkw}, T_{mkw}^c)$
53: **return** $[UT_{mkw}^c, e]$
54: **end Function**

Algorithm 3: D²SSE.Search(K, σ, q, EDB).

1: **Input:** The secret key K, the secret state σ, a query $q = w_{a_1} \vee w_{a_2} \vee \dots \vee w_{a_q}$ and the encrypted database EDB

2: **Output:** Query result R

3: *Client:*

1: $\bigvee_{t=1}^{n_B} q_{mkw}^{(t)} \leftarrow \bigvee_{t=1}^{n_B} \bigwedge_{k=0}^{q_t} mkw_{i_k, j_k}^{(t)}$

2: **for** $t \in [n_B]$ **do**

3: Abbreviate $q_{mkw}^{(t)}$ as $\bigwedge_{k=1}^{\ell} mkw_k$ in this round

4: $c_1 \leftarrow \mathbf{ACT}[mkw_1], c_2 \leftarrow \mathbf{DCT}[mkw_1]$

5: Skip current round if $c_1 = \perp$

6: $K'_{mkw_1} || \tilde{K}'_{mkw_1} \leftarrow F(K'_E, mkw_1)$

7: $K''_{mkw_1} || \tilde{K}''_{mkw_1} \leftarrow F(K''_E, mkw_1)$

8: $ck_{C_1} \leftarrow \mathbf{CPRF.Con}(K'_{mkw_1}, C_{c_1})$

9: $ck_{C_2} \leftarrow \mathbf{CPRF.Con}(K''_{mkw_1}, C_{c_2})$
 $\triangleright C_c$ is the circuit evaluating to 1 on $[0, c]$

10: $a \leftarrow (\tilde{K}'_{mkw_1}, ck_{C_1}, c_1), d \leftarrow (\tilde{K}''_{mkw_1}, ck_{C_2}, c_2)$

11: $sk \leftarrow \mathbf{SK}[mkw_1], \mathbf{SK}[mkw_1] \leftarrow \mathbf{IPE.KeyGen}(1^\lambda)$

12: **for** $i \in [0, c_1]$ **do**

13: **for** $j \in [2, \ell]$ **do**

14: $qtag \leftarrow F(K_X, mkw_1 || mkw_j || i)$

15: $\mathbf{QSet}_i \leftarrow \mathbf{QSet}_i \cup \{qtag\}$

16: **end for**

17: $xtoken_i \leftarrow \mathbf{S^2PE.KeyGen}(msk, \mathbf{QSet}_i)$

18: **end for**

19: Send $token = ((a, d, sk), \{xtoken_i\}_{i=0}^{c_1})$ to server

Server:

20: **for** $i = c_1$ to 0 **do**

21: $T_i \leftarrow \mathbf{CPRF.CEval}(ck_{C_1}, i)$

22: $UT_i \leftarrow H_1(\tilde{K}'_{mkw_1}, T_i)$

23: $(cip_i, tag_i^{add}) \leftarrow \mathbf{EDB}_T[UT_i]$

24: **end for**

25: **for** $j = c_2$ to 0 **do**

26: $T_j \leftarrow \mathbf{CPRF.CEval}(ck_{C_2}, j)$

27: $UT_j \leftarrow H_1(\tilde{K}''_{mkw_1}, T_j)$

28: $(sk_j, tag_j^{del}) \leftarrow \mathbf{EDB}_D[UT_j]$

29: **end for**

30: Server decrypts the ciphertexts $\{(cip_i, tag_i^{add})\}$ with $(sk, \{sk_j\})$ to obtain the list $CInd = \{(e_m || loc_m, tag_m)\}$

31: **for each** $(e_i || loc_i, tag_i) \in CInd$ **do**

32: $y_i || xind_i \leftarrow e_i \oplus H_2(\tilde{K}'_{mkw_1}, T_i)$

33: $ct_i \leftarrow \mathbf{EDB}_X[loc_i]$

34: $msg \leftarrow \mathbf{S^2PE.Dec}(xtoken_i, ct_i)$

35: **if** $msg = \text{"True"}$ **then**

36: Filter out repeated $xind_i$

37: $\mathbf{NewER} \leftarrow \mathbf{NewER} \cup \{y_i\}$

38: **end if**

39: **end for**

40: $\mathbf{OldER} \leftarrow \mathbf{EDB}_{cache}[\tilde{K}'_{mkw_1}]$

41: Remove from OldER indices whose tags are in $\{tag_j^{del}\}$

42: $\mathbf{ER} \leftarrow \mathbf{OldER} \cup \mathbf{NewER}, \mathbf{EDB}_{cache}[\tilde{K}'_{mkw_1}] \leftarrow \mathbf{ER}$

43: Send ER to Client

Client:

44: **for each** $y \in \mathbf{ER}$ **do**

45: $v_1 || v_2 || v_3 \leftarrow \mathbf{SE.Dec}(K_{mkw_1}, y)$

46: $\mathbf{R} \leftarrow \mathbf{R} \cup \{v_2\}$ if $v_3 = 1$

47: **end for**

48: **end for**

49: **return** R

yielding $\max_{i \in [n]} \|w_i\| = \frac{n(n-1)}{2} + 1$. Padding all updates to this bound ensures identical update volumes, thereby hiding keyword-dependent volume patterns. For each $mkw \in M'$, the client computes the update tag and retrieves the corresponding key from **SK** for **IPE.Enc** and **IPE.IncPunc** (lines 6-10).

Once the padded set M' is prepared, the client continues by performing addition or deletion as required. For additions, the client retrieves the addition counter from **ACT**, computes location loc , and invokes **TUpd** (lines 41-54) to obtain address UT_{mkw}^c and value e . The client then binds $e || loc$ to tag using **IPE.Enc** (lines 12-18). The pair $[UT_{mkw}^c, e_T]$ is sent to the server and stored in $\mathbf{EDB}_T$, enabling later retrieval of candidate ciphertexts corresponding to the least-frequent meta-keyword in a conjunctive meta-keyword query. For deletions, the client retrieves the deletion counter from **DCT**, invokes **TUpd**, and calls **IPE.IncPunc** to puncture the decryption key associated with the tag tag . The punctured key, together with its tag, is sent to the server and recorded in $\mathbf{EDB}_D$ (lines 25-34). During subsequent searches, entries fetched from $\mathbf{EDB}_T$ are filtered against $\mathbf{EDB}_D$ to remove deleted documents, leaving only those identifiers of documents that were added, match the least-frequent meta-keyword, and remain undeleted.

Finally, to check whether a surviving document also matches the remaining meta-keywords in a conjunctive meta-keyword query, the client computes a co-occurrence indicator $xtag$ for each other meta-keyword $mkw' \in M$ and inserts these indicators into a co-occurrence set **XSet** (lines 19-22). This set is then encoded using **S²PE.Enc** and uploaded to the server. It will be stored in $\mathbf{EDB}_X$ together with its location loc , enabling private membership tests on candidate documents during subsequent retrieval.

Search: The search protocol, described in Algorithm 3, proceeds in three main stages: i) transforming the disjunctive query into its meta-keyword representation, ii) generating the search token and retrieving candidate documents based on additions and deletions, and iii) verifying co-occurrence with the remaining meta-keywords. Given a disjunctive query $q = w_{a_1} \vee \dots \vee w_{a_q}$, the client first computes the corresponding meta-keyword query $q_{mkw} = \bigvee_{t=1}^{n_B} q_{mkw}^{(t)}$, where $q_{mkw}^{(t)} = mkw_{i_0, j_0}^{(t)} \wedge \dots \wedge mkw_{i_{q_t}, j_{q_t}}^{(t)}$ (line 1). After that, for each conjunctive meta-keyword query (denoted for brevity as $mkw_1 \wedge \dots \wedge mkw_\ell$, where mkw_1 is the least frequent meta-keyword in the corresponding bucket), the client then generates the search token, which consists of two parts: the first part, (a, d, sk) , is used to retrieve document identifiers matching mkw_1 from $\mathbf{EDB}_T$ and $\mathbf{EDB}_D$ (lines 4-11), and the second part $\{xtoken_i\}_{i \in [0, c_1]}$ is generated for retrieving $\mathbf{EDB}_X$ to determine whether each candidate document also matches the remaining meta-keywords (lines 12-19).

Upon receiving (a, d, sk) , the server evaluates the CPRF over $\mathbf{EDB}_T$ and $\mathbf{EDB}_D$ to derive candidate ciphertexts and decrypts them to form the undeleted document list $CInd$ (lines 20-30). The token $\{xtoken_i\}_{i \in [0, c_1]}$ are then used to test whether each candidate also matches the remaining meta-keywords (lines 31-39). During this process, duplicate $xind_i$ values may arise

TABLE I
COMPARISON BETWEEN DIEX [13] AND D²SSE WHEN PERFORMING DISJUNCTIVE KEYWORD QUERY $q = w_{\ell_1} \vee w_{\ell_2} \vee \dots \vee w_{\ell_q}$

Schemes		DIEX [13]	D ² SSE
Storage Overhead	Client-Side	$O\left(N + \sum_{i=1}^N \text{co}(w_i) \right)$	$O(nN)$
	Server-Side	$O\left(\text{DB} + \sum_{i=1}^N \left(\sum_{w \in \text{co}(w_i)} \text{DB}(w_i) \cap \text{DB}(w) \right)\right)$	$O(n^2 \cdot \text{DB})$
Computation Overhead	Update	$O\left((\text{Prf} + \text{H} + \text{Xor} + \text{Enc} + \text{CPrf}_{\text{Eval}}) \cdot W_{\text{id}} ^2\right)$	$O\left(n^2 \cdot (\text{Prf} + \text{H} + \text{Xor} + \text{Enc} + \text{Exp} + \text{BiPa} + \text{CPrf}_{\text{Eval}}) \cdot W_{\text{id}} \right)$
	Search	$O\left(q^2 \cdot (\text{Prf} + \text{H} + \text{Xor} + \text{Dec} + \text{CPrf}_{\text{Eval}}) \cdot \max_{i \in [q]} \text{DB}(w_{\ell_i}) \right)$	$O\left(\sum_{t=1}^{n_B} (\text{Prf} + \text{H} + q \cdot \text{Xor} + \text{Dec} + \text{Exp} + \text{BiPa} + \text{CPrf}_{\text{Eval}}) \cdot \text{DB}(mkw_1^{(t)}) \right)$

Note: $N = |\mathcal{W}|$, n denotes the buckets size, and $n_B = \lceil N/n \rceil$. Prf is the PRF operation, H is the hash function operation, Xor is the exclusive-or operation, Enc and Dec are the SE.Enc and SE.Dec algorithm for SE, Exp is the modular exponentiation, BiPa is the bilinear pairing operation, and $\text{CPrf}_{\text{Eval}}$ and $\text{CPrf}_{\text{Eval}}$ are the CPRF.Eval and CPRF.CEval algorithms of CPRF, respectively. Let $q_{mkw}^{(t)}$, $t \in [n_B]$ denote the conjunctive meta-keyword queries corresponding to q and $mkw_1^{(t)}$ denotes the least frequent meta-keyword in $q_{mkw}^{(t)}$.

because the same document can be associated with multiple keywords that share overlapping meta-keywords. Therefore, line 36 filters duplicate entries to ensure that each document is returned only once, thereby preventing redundant communication overhead. The server then returns the deduplicated results to the client for final decryption (lines 40–49).

B. Theoretical Complexity Analysis

We now provide a theoretical complexity analysis of D²SSE and compare it with the state-of-the-art dynamic scheme DIEX [13]. Our analysis focuses on client-side and server-side storage overhead, as well as the computational complexity of update and search operations under disjunctive keyword queries. In DIEX, both the forward-secure multi-map and the dictionary encryption scheme are instantiated using the FS-RCPRF primitive [9]. A concise summary of the comparison is presented in Table I.

Storage Complexity: On the client side, DIEX stores a secret key and an update counter for each keyword, together with additional counters for all co-occurring keyword pairs. This results in storage complexity $O(N + \sum_{i=1}^N |\text{co}(w_i)|)$, where N denotes the number of keywords and $\text{co}(w_i)$ is the set of keywords co-occurring with w_i . In contrast, D²SSE maintains secret keys and update counters only for meta-keywords. Its client storage complexity is therefore $O(n_B \cdot n^2) = O(nN)$, where n_B and n denote the number of buckets and bucket size, respectively. On the server side, DIEX maintains a global encrypted multi-map for keyword–document pairs and additional local encrypted multi-maps for pairwise keyword intersections. The resulting storage cost is $O(|\text{DB}| + \sum_{i=1}^N \sum_{w \in \text{co}(w_i)} |\text{DB}(w_i) \cap \text{DB}(w)|)$. As for D²SSE, it generates IPE and S²PE ciphertexts for each meta-keyword–“keyword|id” pair, yielding server storage $O\left(\frac{n(n-1)}{2} + 1\right) \cdot |\text{DB}| = O(n^2 |\text{DB}|)$. When n is fixed, this storage grows linearly with database size and remains insensitive to intersection sparsity.

Computational Complexity: Since the setup phase is executed only once during system setup, we focus on update and search operations. Let W_{id} denote the keyword set associated with document identifier id . In DIEX, updating the global multi-map requires $|W_{\text{id}}|$ FS-RCPRF evaluations, while maintaining local intersection maps introduces an additional $O(|W_{\text{id}}|^2)$ invocations. Each invocation involves PRFs, hashing,

XOR, symmetric encryption, and CPRF evaluation, resulting in update complexity $O((\text{Prf} + \text{H} + \text{Xor} + \text{Enc} + \text{CPrf}_{\text{Eval}}) \cdot |W_{\text{id}}|^2)$. For D²SSE, it performs updates per keyword–document pair using PRFs, hashing, XOR, CPRF evaluation, and encryptions under IPE, SE, and S²PE. Since IPE encryption additionally requires modular exponentiation and bilinear pairing, the update complexity becomes $O(n^2 \cdot (\text{Prf} + \text{H} + \text{Xor} + \text{Enc} + \text{Exp} + \text{BiPa} + \text{CPrf}_{\text{Eval}}) \cdot |W_{\text{id}}|)$.

In the search aspect, we consider a disjunctive query $q = w_{\ell_1} \vee \dots \vee w_{\ell_q}$. In DIEX, the server retrieves documents for each queried keyword and eliminates duplicates via pairwise intersection checks. The FS-RCPRF search protocol is therefore invoked once per keyword and once per keyword pair, yielding $O(q^2)$ invocations. The resulting search complexity is $O(q^2 (\text{Prf} + \text{H} + \text{Xor} + \text{Dec} + \text{CPrf}_{\text{Eval}}) \cdot \max_{i \in [q]} |\text{DB}(w_{\ell_i})|)$. In contrast, D²SSE maps q into conjunctive meta-keyword queries $q_{mkw}^{(t)}$, $t \in [n_B]$. For each query, the server retrieves candidate documents associated with the least frequent meta-keyword $mkw_1^{(t)}$, removes deleted entries via IPE decryption, and verifies remaining matches through S²PE. The resulting search complexity is $O(\sum_{t=1}^{n_B} (\text{Prf} + \text{H} + q \cdot \text{Xor} + \text{Dec} + \text{Exp} + \text{BiPa} + \text{CPrf}_{\text{Eval}}) \cdot |\text{DB}(mkw_1^{(t)})|)$.

VI. SECURITY ANALYSIS

In the following, we will show that the new construction above achieves the target security goal, namely, that the adversary cannot learn anything except the pre-defined leakage.

We start by reviewing the leakage at each stage. For clarity, we first focus on the leakage function within a single bucket, and then the overall leakage is defined as the union of the leakage from all buckets. Following the previous notion, we use $\mathcal{L} = \{\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Update}}, \mathcal{L}_{\text{Search}}\}$ of our scheme D²SSE to denote the leakage revealed in the setup, update and search procedures, respectively. It is clear that $\mathcal{L}_{\text{Setup}} = \perp$ because no information is leaked during the setup phase. For the update stage, we have $\mathcal{L}_{\text{Update}}(\text{op}, (w, \text{id})) = \text{op}$, meaning that the only leakage is the type of update operation. Finally, for the search stage, since disjunctive keyword queries are reduced to conjunctive meta-keyword queries, we derive the keyword level leakage from the meta-keyword level in reverse, thereby obtaining the search leakage function $\mathcal{L}_{\text{Search}}$ defined in Section III-B.

Let $\{q^{(i)} = w_1^{(i)} \vee w_2^{(i)} \vee \dots \vee w_d^{(i)}\}_{i \in [Q]}$ denote a sequence of Q disjunctive queries within one bucket, let $\{q_{mkw}^{(i)} = mkw_1^{(i)} \wedge mkw_2^{(i)} \wedge \dots \wedge mkw_\ell^{(i)}\}_{i \in [Q]}$ be the corresponding conjunctive meta-keyword queries. Since the transformation from disjunctive keyword queries to conjunctive meta-keyword queries is performed entirely on the client side and depends only on the relative ordering of keywords within each bucket, an adversary cannot learn the actual keyword values. Consequently, if two conjunctive meta-keyword queries share a common meta-keyword, the adversary can *only* infer that the corresponding disjunctive keyword queries contain the same two keywords, which are not adjacent in the universe $\mathcal{W}$, without learning their exact values. Based on this observation, the detailed derivation of $\mathcal{L}_{\text{Search}}$ is given below.

- $\text{qrp}(q) = (\text{TimeDB}(q), \text{kprp}(q) = \perp)$. $\text{TimeDB}(q)$ is a subset of the search result of the conjunctive meta-keyword query, where each document is paired with its insertion timestamp μ . Moreover, $\text{kprp}(q) = \perp$. The reason is that, given the decryption token x_{token_i} for QSet_i and the ciphertext ct_X for XSet , the server can merely verify whether $\text{QSet}_i \subseteq \text{XSet}$ holds; if it does not, the server learns nothing about the set elements. This conceals Meta-KPRP and thus further hides the KPRP.
- $\text{DelHist}(q)$ is a subset of $\text{DelHist}^{\text{meta}}$, where $\text{DelHist}^{\text{meta}}$, constructible from EDB_T and EDB_D , denotes the set of all deletion timestamps associated with the least frequent meta-keyword in the conjunctive meta-keyword query, along with the insertion timestamps of the corresponding entries that were deleted.
- qop can be derived from ssp^{meta} and csp^{meta} , where ssp^{meta} records which conjunctive meta-keyword queries share the same least-frequent meta-keyword, and csp^{meta} indicates whether two queries not only share that least-frequent meta-keyword but also have an additional common meta-keyword.

Based on the above leakage functions, Theorem 1 establishes the security of the proposed D^2SSE .

Theorem 1 (Adaptive Forward and Backward Privacy of D^2SSE): Let F be a secure PRF and CPRF be a secure CPRF with respect to the circuit family $\mathcal{C} = \{C_c \mid C_c(x) = 1 \Leftrightarrow x \in [0, c]\}$. Assume H_1, H_2 are modeled as random oracles, SE is IND-CPA secure, IPE is IND-PUN-CPA secure, and S^2PE is $\mathcal{L}$ -selectively-secure. Define the leakage function $\mathcal{L} = (\mathcal{L}_{\text{Setup}}, \mathcal{L}_{\text{Update}}, \mathcal{L}_{\text{Search}})$ as previously described. Then, D^2SSE achieves $\mathcal{L}$ -adaptive forward and backward privacy.

Proof: We conduct the proof through a sequence of games from $\text{D}^2\text{SSE}_{\text{Real}}$ to $\text{D}^2\text{SSE}_{\text{Ideal}}$, and we argue that the adversary $\mathcal{A}$ cannot distinguish between any two consecutive games.

Game G_0 : This game follows the same procedures as the original D^2SSE scheme (see Algorithm 1-3). Then, we have

$$\Pr[\text{D}^2\text{SSE}_{\text{Real}}^{\mathcal{A}}(\lambda) = 1] = \Pr[G_0 = 1].$$

Game G_1 : In this game, we replace the keyed PRF F with truly random function. For each new input $mkw||w||\text{id}$ (or mkw , or $mkw||mkw'||c$, or $mkw||c$, or id), a random value is sampled

uniformly at random and stored in a simulated lookup table. For subsequent queries involving the same input, the value is retrieved from the corresponding table. If the adversary $\mathcal{A}$ can distinguish G_1 from G_0 , we can construct an adversary $\mathcal{B}_1$, such that

$$|\Pr[G_0 = 1] - \Pr[G_1 = 1]| \leq 5\text{Adv}_{F, \mathcal{B}_1}^{\text{PRF}}(\lambda).$$

Game G_2 : In this game, we replace the input $w||\text{id}||\text{sig}$ of SE.Enc with constant 0. If an adversary $\mathcal{A}$ can distinguish G_2 from G_1 , then we can construct an adversary $\mathcal{B}_2$ that breaks the IND-CPA security of the underlying symmetric encryption scheme SE. Therefore,

$$|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \text{Adv}_{\text{SE}, \mathcal{B}_2}^{\text{IND-CPA}}(\lambda).$$

Game G_3 : This game is similar G_2 , except that we replace the document identifiers of deleted entries with constant 0 during encryption under the puncturable encryption scheme IPE. The IND-PUN-CPA security of IPE guarantees that G_3 and G_2 are computationally indistinguishable. So there exist an adversary $\mathcal{B}_3$ such that

$$|\Pr[G_2 = 1] - \Pr[G_3 = 1]| \leq \text{Adv}_{\text{IPE}, \mathcal{B}_3}^{\text{PUN-CPA}}(\lambda).$$

Game G_4 : For the algorithms of S^2PE [20], we can use the same technique to simulate the corresponding values by using the leakages defined in $\mathcal{L}$. If the adversary $\mathcal{A}$ can distinguish G_4 from G_3 , then we can construct an adversary $\mathcal{B}_4$ that breaks the $\mathcal{L}$ -selective simulation-based security of S^2PE . Therefore,

$$|\Pr[G_3 = 1] - \Pr[G_4 = 1]| \leq \text{Adv}_{\mathcal{L}, \mathcal{B}_4}^{\text{S}^2\text{PE}}(\lambda).$$

Game G_5 : For CPRF and hash functions H_1, H_2 , we can simulate the corresponding values using the same technique as in [9]. Therefore, there exists a reduction $\mathcal{B}_5$ such that

$$|\Pr[G_4 = 1] - \Pr[G_5 = 1]| \leq 2P \cdot \text{Adv}_{\mathcal{B}_5}^{\text{CPRF}}(\lambda) + \frac{2Pq}{2^\lambda},$$

where P is the number of encrypted entries associated with meta-keywords in the database and q is the number of queries.

Simulator: The query overlap pattern qop enables the simulation of search queries. Furthermore, the encrypted database EDB can be properly simulated by leveraging the leakage functions $\text{TimeDB}(q)$ and $\text{DelHist}(q)$ as inputs of $\text{D}^2\text{SSE.Search}$. Based on this, we conclude that Game G_5 can be simulated by the simulator $\mathcal{S}$ under the defined leakage functions. Consequently, we obtain

$$\Pr[G_5 = 1] = \Pr[\text{D}^2\text{SSE}_{\text{Ideal}}^{\mathcal{A}, \mathcal{S}}(\lambda) = 1].$$

In summary, the advantage of any adversary $\mathcal{A}$ in attacking D^2SSE is given by

$$\begin{aligned} & |\Pr[\text{D}^2\text{SSE}_{\text{Real}}^{\mathcal{A}}(\lambda) = 1] - \Pr[\text{D}^2\text{SSE}_{\text{Ideal}}^{\mathcal{A}, \mathcal{S}}(\lambda) = 1]| \\ & \leq 5\text{Adv}_{F, \mathcal{B}_1}^{\text{PRF}}(\lambda) + \text{Adv}_{\text{SE}, \mathcal{B}_2}^{\text{IND-CPA}}(\lambda) + \text{Adv}_{\text{IPE}, \mathcal{B}_3}^{\text{PUN-CPA}}(\lambda) \\ & \quad + \text{Adv}_{\mathcal{L}, \mathcal{B}_4}^{\text{S}^2\text{PE}}(\lambda) + 2P \cdot \text{Adv}_{\mathcal{B}_5}^{\text{CPRF}}(\lambda) + \frac{2Pq}{2^\lambda}. \end{aligned}$$

This completes the proof. $\square$

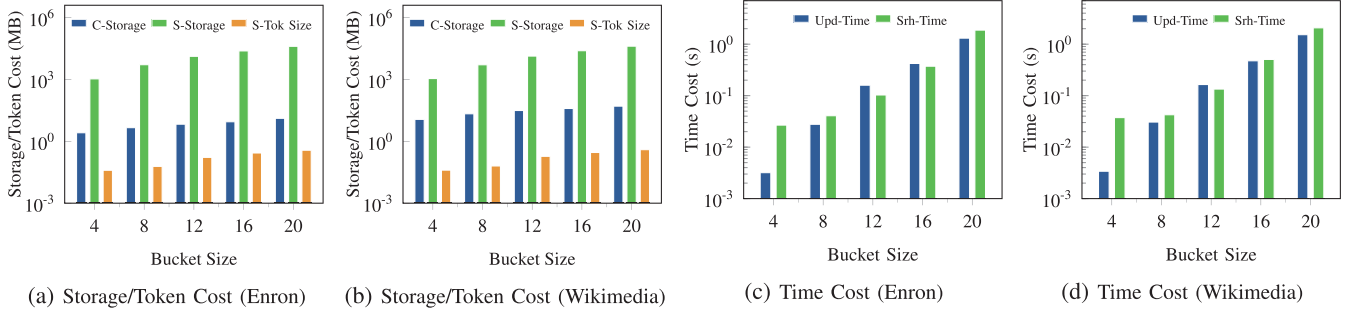


Fig. 1. Impact of bucket size on storage overhead, computation time, and search token size in D^2SSE . We use C-Storage, S-Storage, S-Tok Size, Upd-Time, and Srh-Time to represent client-side storage, server-side storage, search token size, update time, and search time, respectively.

TABLE II
INSERTION SUCCESS RATE OF XORFILTER UNDER DIFFERENT BUCKET SIZES
AND FILTER LENGTHS, WHERE n' IS THE INPUT SET SIZE

XorFilter Length b	Bucket Size				
	4	8	12	16	20
$b = \lfloor 1.23 \cdot n' \rfloor + 16$	97.2%	89.1%	86.6%	80.3%	77.9%
$b = \lfloor 1.23 \cdot n' \rfloor + 32$	98.7%	97.3%	94.8%	93.1%	93.9%
$b = \lfloor 1.23 \cdot n' \rfloor + 64$	100%	98.5%	97.0%	97.1%	96.7%

VII. PERFORMANCE EVALUATIONS

In this section, we conduct a detailed experimental evaluation of our refined scheme D^2SSE , and compare it with the state-of-the-art $DIEX$ [13] and the toy construction $TWIN_{ODXT}$.

A. Implementation Setup

We implement D^2SSE and all compared schemes in C++ and conduct comprehensive simulations on a laptop equipped with an Apple M2 Pro chip and 32 GB RAM. For cryptographic primitives, we use the Crypto++ library, instantiating AES-256 in CBC mode for symmetric encryption, BLAKE2b for hashing, and HMAC-BLAKE2b for pseudorandom functions. We instantiate the S^2PE component using XorFilter with a false-positive rate of 10^{-6} in D^2SSE . All experiments are conducted on two real-world datasets: the Enron Email Dataset¹ and the Wikimedia Database².

Parameter Choices: To balance performance and functionality, we tune two key parameters in D^2SSE , namely the XorFilter length and the bucket size. The XorFilter length affects insertion success probability and ciphertext size, while the bucket size influences search token size, ciphertext overhead, and the efficiency of update and search operations. Since the XorFilter encodes the set $XSet$ of size $n(n-1)/2$, its required length depends on the bucket size n . We therefore evaluate insertion success under different bucket sizes and candidate length parameters. Following the guideline in [26] and the empirical results in Table II, we set the length parameter to $b = \lfloor 1.23n' \rfloor + 32$, which achieves high insertion success while keeping ciphertext expansion moderate. Regarding the bucket size, we randomly sample 10^5 keyword-identifier pairs from the datasets and measure system overhead under varying configurations. As shown

in Fig. 1, overhead increases with the bucket size, indicating that smaller buckets yield better performance. However, excessively small buckets weaken disjunctive aggregation; in the extreme case where $n = 1$, disjunctive queries degenerate into multiple independent single-keyword searches. To balance efficiency and query functionality, we set $n = 8$ for all subsequent experiments.

B. Evaluation Results

Using these configurations, we then perform a comprehensive comparison among $DIEX$, $TWIN_{ODXT}$, and D^2SSE across three key dimensions: computation (update time and search latency), storage (client-side and server-side), and communication (search token size and the per query result cost). The detailed results are presented below.

Computation Performance: We first evaluate the computational cost of the compared schemes by examining their performance in both update and search operations.

In terms of update efficiency, we evaluate update performance by running the update protocols of $DIEX$, $TWIN_{ODXT}$, and our D^2SSE on datasets of varying sizes. As shown in Fig. 2(a) and (b), D^2SSE exhibits an update time approximately $1.8\times$ higher than that of $DIEX$ and $1.6\times$ higher than that of $TWIN_{ODXT}$. This overhead mainly stems from constructing a small XorFilter during each update, which is essential for mitigating Meta-KPRP leakage. Nevertheless, even when the dataset size reaches 10^6 , the update latency per operation remains only 29 ms, indicating that D^2SSE is still highly practical.

We then assess the search efficiency of different schemes under varying query lengths. Using a fixed dataset of size 10^5 , we issue disjunctive queries of lengths 5, 10, 15, 20, and 25, each returning exactly 100 documents. The results in Fig. 2(c) and (d) show that D^2SSE consistently outperforms $TWIN_{ODXT}$. In particular, for queries of length 25, D^2SSE is up to $13\times$ faster than $TWIN_{ODXT}$. Compared with $DIEX$, D^2SSE is slightly slower when the query length is at most 20, but an encouraging trend emerges as the query length grows: once it exceeds 20, D^2SSE begins to surpass $DIEX$, and at length 25 it is roughly $0.25\times$ faster. This scalability advantage comes from the

¹Enron Email Dataset: <https://www.cs.cmu.edu/~enron/>

²Wikimedia Database: <https://dumps.wikimedia.org/>

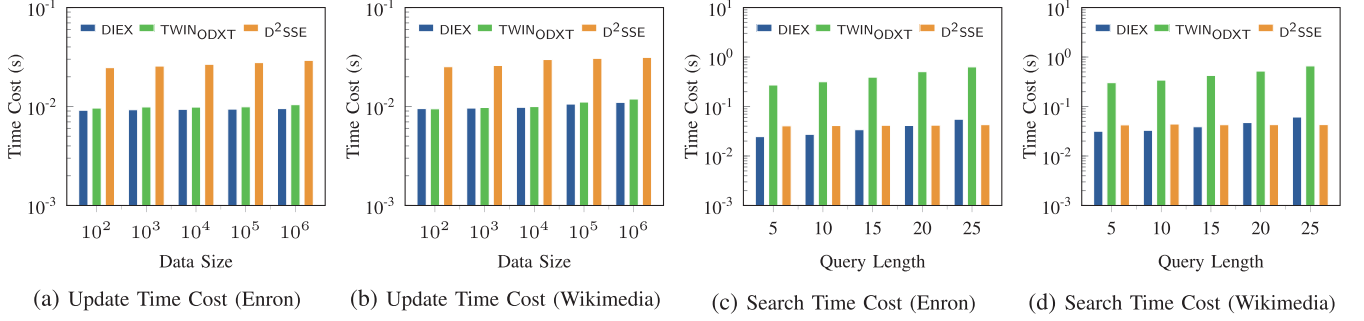


Fig. 2. Comparison of update and search time costs between prior works and our approach on the Enron Email and Wikimedia Datasets.

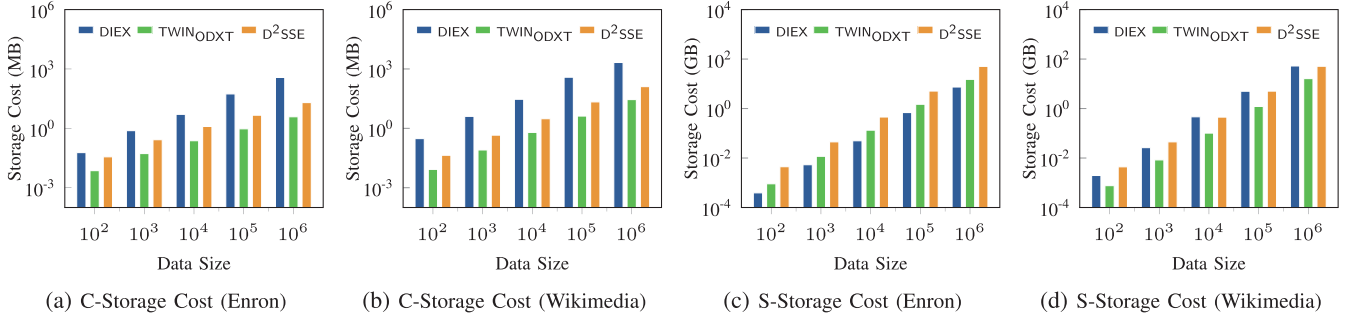


Fig. 3. Comparison of storage overhead between prior works and our approach on the Enron Email and Wikimedia Datasets, where C-Storage and S-Storage represent client-side storage and server-side storage, respectively.

use of XorFilter, which compactly encodes meta-keyword co-occurrence and enables efficient retrieval via lightweight XOR operations, whereas DIEX requires explicit pairwise retrieval and deduplication, which becomes increasingly costly for longer queries.

Storage Performance: We next evaluate the storage overhead of different schemes from both the client and server perspectives. On the client side, as shown in Fig. 3(a) and (b), D²SSE incurs approximately 0.06 $\times$ the storage overhead of DIEX and 4.5 $\times$ that of TWIN_{ODXT} when the dataset size reaches 10⁶. The substantially lower cost compared with DIEX is due to the fact that DIEX must maintain update counters for every keyword pair co-occurring in the same document to support dynamic updates over its local EMMs, which significantly enlarges client storage. In contrast, the higher overhead relative to TWIN_{ODXT} is attributed to the additional deletion counters and puncturable encryption keys (i.e., **DCT** and **SK**) maintained by D²SSE for each meta-keyword.

On the server side, storage overhead is strongly influenced by the underlying database distribution. As shown in Fig. 3(c) and (d), for the Enron Email dataset, where intersections among keyword document sets are sparse, D²SSE incurs approximately 7.2 $\times$ the storage overhead of DIEX. In contrast, for the Wikimedia dataset, where such intersections are substantially denser, the storage overhead of D²SSE becomes nearly comparable to that of DIEX. This difference arises from their intersection encoding strategies. Specifically, D²SSE employs fixed-size XorFilters, making storage depend primarily on the number of keywords, whereas DIEX relies on encrypted multi-maps whose size scales with the number of

intersections. Furthermore, compared with TWIN_{ODXT}, D²SSE incurs about 3.1 $\times$ higher storage overhead due to XorFilter, which enables private set membership testing and effectively conceals Meta-KPRP leakage. Although D²SSE requires more storage than TWIN_{ODXT} and DIEX, the overhead arises from XorFilter-based encoding that strengthens privacy, especially by hiding Meta-KPRP leakage. Considering the scalability and low cost of cloud storage, this tradeoff remains practical.

Communication Performance: Finally, we analyze the communication overhead of D²SSE, which consists of two parts: the size of the search token sent by the client during the search phase and the per query result cost, defined as the ratio of irrelevant documents to relevant documents among all returned results. The latter is evaluated because D²SSE employs the TWINSSE framework, which not only returns all relevant documents but also a fraction of irrelevant ones, thereby incurring extra communication cost.

As shown in Fig. 4(a) and (b), the results indicate that the search token size of TWIN_{ODXT} and DIEX increases with the query length and eventually surpasses that of D²SSE. When the query length reaches 25, the search token size of D²SSE is only 0.8 $\times$ that of DIEX and 0.7 $\times$ that of TWIN_{ODXT}. This is because, after retrieving the documents matched with the least frequent meta-keyword, D²SSE can use a fixed-size token to jointly test whether each candidate document matches the remaining meta-keywords, and thus the search token size does not grow with the query length.

As for the communication cost per query result, we measure it across data sizes ranging from 10³ to 10⁶ keyword-identifier pairs. As shown in Fig. 4(c) and (d), D²SSE consistently

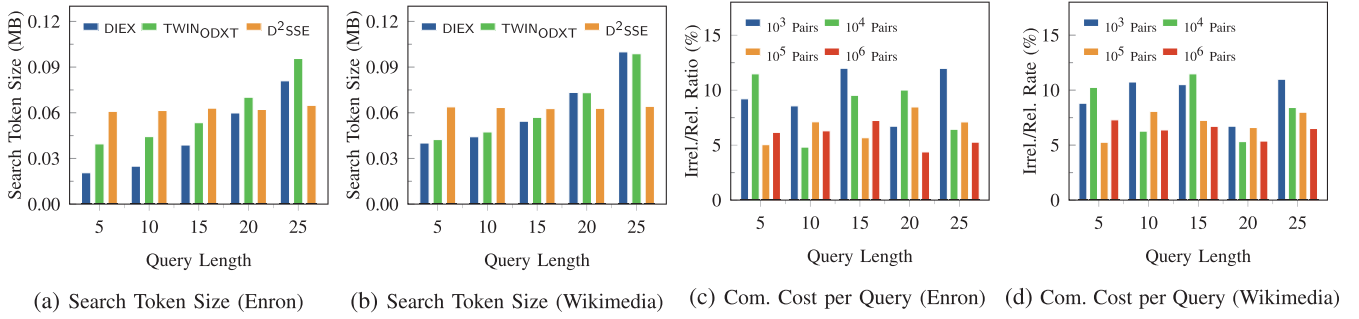


Fig. 4. Comparison of search token size with prior works and evaluation of communication (i.e., Com.) cost per query result of our scheme on Enron Email and Wikimedia datasets, where Irrel. and Rel. are irrelevant and relevant documents, respectively.

maintains an irrelevant-to-relevant document ratio below 12% across both datasets, regardless of query length or database size. By comparison, DIEX guarantees a ratio of 0% but leaks KPRP and does not ensure backward privacy, highlighting the trade-off between privacy and accuracy.

VIII. RELATED WORK

(Dynamic) Searchable Symmetric Encryption: SSE was first introduced by Song et al. [2]. Curtmola et al. [3] later formalized its adaptive security and proposed an efficient indexed construction. Since then, extensive efforts have been devoted to enhancing security guarantees [27], [28], [29], improving search efficiency [30], [31], [32], and enriching query expressiveness [17], [33], [34]. To further improve functionality, researchers have explored more expressive encrypted query primitives and verification mechanisms. Most recently, Li et al. [35] proposed encrypted conjunctive geometric range queries with learned index support, while Ji et al. [36] developed a verifiable conjunctive search framework based on additive homomorphic encryption to ensure result correctness.

Beyond the above progress in functionality, the rapid growth of data volumes has motivated recent studies to enhance security for update operations. Bost et al. [8], [9] systematically formalized the notions of forward and backward privacy. Based on these foundations, subsequent studies broadened the type of queries and investigated more expressive search paradigms in dynamic environments [22], [37], [38], [39], [40], [41]. Meanwhile, recent efforts have increasingly focused on ensuring result verifiability and freshness guarantees. Zhu et al. [42] proposed a generic verifiable DSSE framework based on Merkle Patricia trees and timestamp chains to ensure search-result integrity and freshness under dynamic updates, whereas Hu et al. [43] introduced a linked key span Merkle hash tree to achieve low-overhead freshness authentication with strong real-time guarantees.

SSE with Disjunctive Query: Disjunctive queries, which are crucial for enabling flexible retrieval in practical applications, have received widespread attention. However, early encrypted search schemes that support disjunctive queries were either based on public-key cryptography [44], [45], [46] or relied on fake disjunctive queries—i.e., by repeatedly checking whether

each individual keyword in a disjunctive query exists in a document [47]. To enable efficient disjunctive queries in the symmetric-key setting, Kamara et al. [13] leveraged set-theoretic principles to present the first non-interactive disjunctive SSE scheme, IEX, with worst-case sub-linear complexity. Later, Bag et al. [15] introduced the concept of meta-keywords and proposed the TWINSSE framework, which first reduces disjunctive keyword queries to conjunctive meta-keyword queries and then implements them using existing conjunctive SSE scheme OXT [17]. Building on this framework, Wu et al. [16] further enhanced the query capabilities by proposing a scheme that supports arbitrary Boolean query-join.

Despite the aforementioned advances, the study of disjunctive queries in dynamic settings remains relatively scarce and underexplored. DIEX [13] represents a representative construction supporting this query type, which aims to achieve stronger privacy guarantees for dynamic disjunctive queries. However, despite achieving forward privacy, DIEX neither considers backward privacy nor protects against KPRP leakage.

IX. CONCLUSION

In this paper, we formalize the security model for disjunctive dynamic searchable symmetric encryption (DDSSSE) and propose a new DDSSSE construction, D²SSE, that simultaneously provides both forward and backward privacy, and conceals keyword-pair result pattern leakage, which are security properties not simultaneously achieved by any existing scheme. Finally, we implement D²SSE and conduct comprehensive evaluations comparing it with the state-of-the-art scheme DIEX and TWIN_{ODXT}. Experimental results show that D²SSE is superior to DIEX and TWIN_{ODXT} in terms of search efficiency, while also avoiding the quadratic growth of communication cost with respect to query length.

REFERENCES

- [1] B. Fuller et al., “SoK: Cryptographically protected database search,” in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 172–191.
- [2] D. X. Song, D. A. Wagner, and A. Perrig, “Practical techniques for searches on encrypted data,” in *Proc. IEEE Symp. Secur. Privacy*, 2000, pp. 44–55.
- [3] R. Curtmola, J. A. Garay, S. Kamara, and R. Ostrovsky, “Searchable symmetric encryption: Improved definitions and efficient constructions,” in *Proc. ACM Conf. Comput. Commun. Secur.*, 2006, pp. 79–88.

- [4] D. Cash and S. Tessaro, "The locality of searchable symmetric encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2014, pp. 351–368.
- [5] S. Kamara, C. Papamanthou, and T. Roeder, "Dynamic searchable symmetric encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2012, pp. 965–976.
- [6] D. Cash et al., "Dynamic searchable encryption in very-large databases: Data structures and implementation," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2014.
- [7] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable encryption with small leakage," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2014.
- [8] R. Bost, "Σ_{oφoς}: Forward secure searchable encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2016, pp. 1143–1154.
- [9] R. Bost, B. Minaud, and O. Ohrimenko, "Forward and backward private searchable encryption from constrained cryptographic primitives," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2017, pp. 1465–1482.
- [10] L. Xu, H. Duan, A. Zhou, X. Yuan, and C. Wang, "Interpreting and mitigating leakage-abuse attacks in searchable symmetric encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 5310–5325, 2021.
- [11] L. Xu, L. Zheng, C. Xu, X. Yuan, and C. Wang, "Leakage-abuse attacks against forward and backward private searchable symmetric encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2023, pp. 3003–3017.
- [12] Z. Gui, K. G. Paterson, and S. Patranabis, "Rethinking searchable symmetric encryption," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 1401–1418.
- [13] S. Kamara and T. Moataz, "Boolean searchable symmetric encryption with worst-case sub-linear complexity," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2017, pp. 94–124.
- [14] S. Patel, G. Persiano, J. Y. Seo, and K. Yeo, "Efficient Boolean search over encrypted data with reduced leakage," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2021, pp. 577–607.
- [15] A. Bag, D. Talapatra, A. Rastogi, S. Patranabis, and D. Mukhopadhyay, "TWo-IN-one-SSE: Fast, scalable and storage-efficient searchable symmetric encryption for conjunctive and disjunctive Boolean queries," *Proc. Priv. Enhancing Technol.*, vol. 2023, no. 1, pp. 115–139, 2023.
- [16] J. Wu, K. Zhang, L. Wei, J. Gong, and J. Ning, "Practical searchable symmetric encryption for arbitrary Boolean query-join in cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 10086–10098, 2024.
- [17] D. Cash, S. Jarecki, C. S. Jutla, H. Krawczyk, M. Rosu, and M. Steiner, "Highly-scalable searchable symmetric encryption with support for Boolean queries," in *Proc. Annu. Cryptol. Conf.*, 2013, pp. 353–373.
- [18] Y. Zhang, J. Katz, and C. Papamanthou, "All your queries are belong to us: The power of file-injection attacks on searchable encryption," in *Proc. USENIX Secur. Symp.*, 2016, pp. 707–720.
- [19] S. Lai et al., "Result pattern hiding searchable encryption for conjunctive queries," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2018, pp. 745–762.
- [20] Y. Wang, S. Sun, J. Wang, X. Chen, J. K. Liu, and D. Gu, "Practical non-interactive encrypted conjunctive search with leakage suppression," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2024, pp. 4658–4672.
- [21] S. Patranabis and D. Mukhopadhyay, "Forward and backward private conjunctive searchable symmetric encryption," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2021.
- [22] C. Zuo et al., "Searchable encryption for conjunctive queries with extended forward and backward privacy," *Proc. Priv. Enhancing Technol.*, vol. 2025, no. 1, pp. 440–455, 2025.
- [23] D. Cash, P. Grubbs, J. Perry, and T. Ristenpart, "Leakage-abuse attacks against searchable encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2015, pp. 668–679.
- [24] G. Couteau, P. Meyer, A. Passelègue, and M. Riahinia, "Constrained pseudorandom functions from homomorphic secret sharing," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2023, pp. 194–224.
- [25] S. Kamara and T. Moataz, "Computationally volume-hiding structured encryption," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2019, pp. 183–213.
- [26] T. M. Graf and D. Lemire, "XOR filters: Faster and smaller than Bloom and cuckoo filters," *ACM J. Exp. Algorithmics*, vol. 25, pp. 1–16, 2020.
- [27] G. Chen, T. Lai, M. K. Reiter, and Y. Zhang, "Differentially private access patterns for searchable symmetric encryption," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 810–818.
- [28] Q. Song, Z. Liu, J. Cao, K. Sun, Q. Li, and C. Wang, "SAP-SSE: Protecting search patterns and access patterns in searchable symmetric encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 1795–1809, 2021.
- [29] T. Le, R. Behnia, J. Guajardo, and T. Hoang, "MUSES: Efficient multi-user searchable encrypted database," in *Proc. USENIX Secur. Symp.*, 2024, pp. 2581–2598.
- [30] I. Demertzis, D. Papadopoulos, and C. Papamanthou, "Searchable encryption with optimal locality: Achieving sublogarithmic read efficiency," in *Proc. Annu. Cryptol. Conf.*, 2018, pp. 371–406.
- [31] I. Demertzis, C. Papamanthou, and R. Talapatra, "Efficient searchable encryption through compression," *Proc. VLDB Endowment*, vol. 11, no. 11, pp. 1729–1741, 2018.
- [32] Y. Miao, J. Ma, X. Liu, J. Weng, H. Li, and H. Li, "Lightweight fine-grained search over encrypted data in fog computing," *IEEE Trans. Serv. Comput.*, vol. 12, no. 5, pp. 772–785, Sep./Oct. 2019.
- [33] C. Wang, N. Cao, J. Li, K. Ren, and W. Lou, "Secure ranked keyword search over encrypted cloud data," in *Proc. Int. Conf. Distrib. Comput. Syst.*, 2010, pp. 253–262.
- [34] I. Demertzis, S. Papadopoulos, O. Papapetrou, A. Deligiannakis, and M. N. Garofalakis, "Practical private range search revisited," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2016, pp. 185–198.
- [35] M. Li, C. Jia, R. Du, and G. Ha, "Efficient conjunctive geometric range query over encrypted spatial data with learned index," *IEEE Trans. Comput.*, vol. 74, no. 12, pp. 3995–4009, Dec. 2025.
- [36] L. Ji, J. Li, Y. Zhang, and Y. Lu, "Verifiable searchable symmetric encryption over additive homomorphism," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 1320–1332, 2025.
- [37] C. Guo, W. Li, X. Tang, K. R. Choo, and Y. Liu, "Forward private verifiable dynamic searchable symmetric encryption with efficient conjunctive query," *IEEE Trans. Dependable Secur. Comput.*, vol. 21, no. 2, pp. 746–763, Mar./Apr. 2024.
- [38] Q. Liu, Y. Tian, J. Wu, T. Peng, and G. Wang, "Enabling verifiable and dynamic ranked search over outsourced data," *IEEE Trans. Serv. Comput.*, vol. 15, no. 1, pp. 69–82, Jan./Feb. 2022.
- [39] C. Zuo, S. Sun, J. K. Liu, J. Shao, and J. Pieprzyk, "Dynamic searchable symmetric encryption schemes supporting range queries with forward (and backward) security," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2018, pp. 228–246.
- [40] W. Yang, Y. Geng, L. Li, X. Xie, and L. Huang, "Achieving secure and dynamic range queries over encrypted cloud data," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 107–121, Jan. 2022.
- [41] L. Chen, J. Li, and J. Li, "Toward forward and backward private dynamic searchable symmetric encryption supporting data deduplication and conjunctive queries," *IEEE Internet Things J.*, vol. 10, no. 19, pp. 17408–17423, Oct. 2023.
- [42] J. Zhu, Q. Li, C. Wang, X. Yuan, Q. Wang, and K. Ren, "Enabling generic, verifiable, and secure data search in cloud services," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 8, pp. 1721–1735, Aug. 2018.
- [43] Y. Hu, R. Zhang, and Y. Zhang, "KV-fresh: Freshness authentication for outsourced multi-version key-value stores," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 1638–1647.
- [44] J. Katz, A. Sahai, and B. Waters, "Predicate encryption supporting disjunctions, polynomial equations, and inner products," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2008, pp. 146–162.
- [45] Y. Zhang and S. Lu, "POSTER: Efficient method for disjunctive and conjunctive keyword search over encrypted data," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2014, pp. 1535–1537.
- [46] Z. Lin, H. Li, X. Chen, M. Xiao, and Q. Huang, "Identity-based encryption with disjunctive, conjunctive and range keyword search from lattices," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 8644–8657, 2024.
- [47] T. Moataz and A. Shikfa, "Boolean symmetric searchable encryption," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2013, pp. 265–276.



Xiaojun Zhang received the BE degree from the Nanjing University of Science and Technology, China, in 2023. He is currently working toward the master's degree with the School of Mathematics and Statistics, Nanjing University of Science and Technology, China. His research interests focus on applied cryptography and data privacy protection.



Lei Xu received the PhD degree from the Nanjing University of Science and Technology, 2019. He is currently an associate professor with the School of Mathematics and Statistics, Nanjing University of Science and Technology. Before that, he was a post-doc with the Department of Computer Science, City University of Hong Kong, Hong Kong. He was also ever a visiting PhD student with Monash University during the period from April 2017 to April 2018. His main research interests focus on applied cryptography and information security. In the past few years, his work has appeared in prestigious venues in cybersecurity, computer networks, and distributed systems, such as ACM CCS, USENIX SEC, VLDB, USENIX NDSI, IEEE INFOCOM, ACM ASIACCS, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Computers*, *IEEE Transactions on Services Computing*, etc.



Lin Mei received the BE and PhD degrees from the Nanjing University of Science and Technology, China, in 2017, and 2023, respectively. She is currently with the School of Mathematics and Statistics, Nanjing University of Science and Technology, as a post-doctoral researcher. Before that, she was also a visiting PhD student with the School of Physical and Mathematical Sciences, Nanyang Technological University, from October 2022 to September 2023. Her research interests focus on applied cryptography and data privacy protection.



Xingliang Yuan (Senior Member, IEEE) is an associate professor with the School of Computing and Information Systems, University of Melbourne, and a future fellow of the Australian Research Council (ARC). Previously, he served as a faculty member with the Faculty of IT, Monash University (2017–2024). His research focuses on designing secure networked systems and protocols to protect digital assets in untrusted environments. His research has been supported by ARC, CSIRO, and the Australian Departments of Home Affairs, and Health and Aged

Care. His work is regularly published in major computer security and networked system venues. His contributions have earned him several honours, including the Dean's Award for Excellence in Research for an ECR (2020), the Faculty Teaching Excellence Award (2021), and most recently, the Excellence in Engagement Award with UniMelb (2024). He is a co-recipient of the Best Paper Award with ESORICS (2021) and an Honourable Mention Paper Award with USENIX Security (2025). He serves on the editorial board of *IEEE Transactions on Dependable and Secure Computing* and *IEEE Transactions on Services Computing* (area editor in *Security, Privacy, and Trust*), and as a general chair for ICDCS'27 and RAID'25, a PC chair for Lamps@CCS'24, SecTL@AsiaCCS'23, and NSS'22, and a track chair for ICDCS'24. He has also received the Notable Reviewers award recently with USENIX Security (2025).



Chungen Xu received the MS degree from East China Normal University, Shanghai, China, in 1996, and the PhD degree from the Nanjing University of Science and Technology, in 2003. He is currently a professor with the School of Mathematics and Statistics, Nanjing University of Science and Technology, China. His current interests are in the areas of computer and network security, applied cryptography and coding. His research has been supported by multiple research fund agencies, including the National Natural Science Foundation of China, National Key Research and Development Program of China, the Natural Science Foundation of Jiangsu Province, and the Fundamental Research Funds for the Central Universities, etc.



Yifeng Zheng is an assistant professor with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University. His current research interests are focused on privacy-aware computing, machine learning security and privacy, and secure networked systems. His research works have been published in prestigious venues (such as USENIX Security, CCS, VLDB, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, and *IEEE Transactions on Services Computing*). He received the Distinguished Paper Award from ACM CCS 2025 and the Best Paper Award from ESORICS 2021. He has served on the technical program committees of various international conferences such as USENIX Security, PETS, EuroS&P, AsiaCCS, ICDCS, and IWQoS. He currently serves as associate editor for *IEEE Transactions on Services Computing*.

received the Distinguished Paper Award from ACM CCS 2025 and the Best Paper Award from ESORICS 2021. He has served on the technical program committees of various international conferences such as USENIX Security, PETS, EuroS&P, AsiaCCS, ICDCS, and IWQoS. He currently serves as associate editor for *IEEE Transactions on Services Computing*.